

Chiffrage HET - Documentation projet

Sommaire

- [1. Objectif](#)
- [2. Installation](#)
- [3. Vue d'ensemble fonctionnelle](#)
- [4. Architecture logicielle](#)
- [5. Modele metier et regles de calcul](#)
- [6. Configuration et donnees](#)
- [7. Format des projets sauvegardes](#)
- [8. Exports](#)
- [9. Structure du depot](#)
- [10. Guide de maintenance](#)
- [11. Resume](#)

1. Objectif

Chiffrage HET est une application de bureau PyQt6 destinee au chiffrage des heures d'ingenierie pour les affaires HET.

Elle remplace un classeur Excel par une application qui :

- centralise la definition du projet ;
- calcule les heures automatiquement a partir de regles metier et de coefficients ;
- permet des corrections manuelles a la tache ou a la categorie ;
- produit des exports de rapports Excel et Heures d'études par métier ;
- permet la sauvegarde et l'importation de projet de chiffrage ;
- interroge une base REX de machines historiques.

Le projet est orienté vers une application desktop Windows et s'appuie sur des fichiers JSON contenant des donnees metiers, des templates Excel et une base REX Excel.

2. Installation

Pré-requis

- Python 3.10 ou plus recent
- Windows recommandé
- acces aux dossiers et fichiers references dans `config.yaml`

Dépendances

```
pip install -r requirements.txt
```

- **PyQt6** pour l'interface graphique ;
- **openpyxl** pour les templates et exports Excel ;
- **pandas** pour la base REX ;
- **PyYAML** pour la configuration.

Données

Avant le premier lancement, verifier **config.yaml**.

Par défaut, plusieurs chemins pointent vers un lecteur reseau **S:**. Sans accès réseau, les fonctionnalités liées au partage seront inaccessibles :

- **project-save-dir**: dossier de sauvegarde par défaut
- **asset-dir** : dossier racine pour l'importation de projet
- **quick-export-path**: Exportation rapide sur le réseau
- **rex-database-path**: Base de données commune

Les chemins peuvent être modifiés pour atteindre des fichiers locaux si besoin.

Les chemins des données JSON et des templates Excel sont également définis dans ce fichier et n'ont pas besoin d'être modifiés. Si vous copiez le programme sur votre machine, emportez les dossiers **data**, **assets** et **src**.

Lancement

Lancer depuis la racine du projet :

```
python main.py
```

Compilation

Après mise à jour du code, vous voudrez mettre à jour l'exécutable et les fichiers présents dans le dossier de partage, ou il sera utilisé.

Sur le réseau JE, lancer le fichier **build.bat** pour compiler le programme.

```
start build.bat
```

Ce programme va :

- garder en mémoire la dernière version de la base de données commune (attention : il va écraser la base que vous avez en local si vous en avez une) ;
- nettoyer le dossier du programme dans le lecteur réseau de partage ;
- compiler le programme en un exécutable et le copier, ainsi que tous ses fichiers, dans le dossier de partage ;
- supprimer les fichiers inutiles issus de la compilation.

3. Vue d'ensemble fonctionnelle

L'application est organisee en 6 onglets. Onglet 1 : informations generales Onglets 2, 3 et 4 : modification manuelle des heures d'etude.

- Base : heures calculees en fonction des informations generales
- Heures finales : heures reelles prises en compte dans le total
- L'onglet 4.LPDC contient des coefficients qui dependent des informations generales mais qui peuvent etre modifies.
- Il est possible d'apporter des corrections par categorie. Dans ce cas, les heures supplementaires seront reparties sur chacune des taches de la categorie proportionnellement aux heures de base.

3.1 General

Onglet de saisie du contexte projet :

- numero CRM ;
- revision ;
- date ;
- realise par / valide par ;
- type d'affaire ;
- client ;
- DAS ;
- secteur ;
- categorie produit ;
- produit ;
- designation ;
- nombre de machines ;
- description.

Il est également possible d'importer un projet JSON existant.

Points importants :

- les champs marques d'un * dans l'interface reconstruisent le projet a partir des donnees de reference (attention : les modifications manuelles ne seront pas sauvegardees. Verifiez bien les informations entrees ici avant de passer a la suite) ;
- le type d'affaire met a jour les coefficients dependants de l'affaire ;
- les mises a jour lourdes sont debouncees a 300 ms pour eviter les reconstructions inutiles.

3.2 Definition

Cet onglet regroupe :

- **Enclenchement** ;
- **Calculs** ;
- **Plans / Specs / LDN** ;
- **Suivi**.

Les calculs optionnels apparaissent avec des cases a cocher. Les categories de calculs sont repliees par default.

3.3 Labo et Options

Cet onglet regroupe :

- les taches de laboratoire ;
- les options techniques.

3.4 LPDC

Cet onglet affiche les documents LPDC applicables au contexte courant.

- deux coefficients globaux sont modifiables dans l'onglet : coefficient secteur et coefficient affaire.

3.5 Resume

L'onglet Resume affiche :

- un arbre de synthèse des heures par famille ;
- le sous-total de la premiere machine ;
- le pourcentage Divers risques techniques ;
- le total premiere machine ;
- le total pour n machines ;
- le coefficient REX ou les heures REX equivalentes ;
- le total final ;
- le delai d'etude estime en mois.

Le menu d'export permet :

- export rapide : cree une sauvegarde, un rapport Excel et un chiffrage par metier `prepa_ORTEMS`, puis les place dans les dossiers par defaut avec les noms par defaut en un clic ;
- sauvegarde JSON ;
- export Excel ORTEMS ;
- export Excel rapport de chiffrage.

Un bouton d'engrenage ouvre une boite de dialogue de parametrage des valeurs qui entrent dans le calcul du delai d'etude. Les valeurs modifiees sont sauvegardees dans `data/base_data.json` et seront utilisees dans les prochains chiffrages.

3.6 Recherche REX

Cet onglet charge une base Excel de machines historiques et permet une recherche par :

- texte libre ;
- listes deroulantes ;
- valeurs numeriques avec tolerance ;
- filtres dedies a l'IP et au nombre de poles.

Fonctionnalites notables :

- la recherche conserve les lignes dont certaines colonnes filtrees sont vides ;
- le double-clic sur un resultat ouvre le detail du projet ;

- le detail permet l'edition directe de certaines cellules ;
 - les modifications sont ecrites dans le fichier Excel source de la base REX.
-

4. Architecture logicielle

L'application suit une architecture MVC simple avec signaux Qt.

4.1 Point d'entree

`main.py` :

- cree `ApplicationData` ;
- appelle `sort_raw_data()` ;
- cree `QApplication` ;
- instancie `Controller`.

4.2 Controller principal

`src/controller.py` :

- cree `Model` et `MainWindow` ;
- instancie les 6 onglets ;
- connecte l'import et les exports ;
- affiche la fenetre principale.

Onglets instances :

- `TabGeneral` ;
- `DefinitionTabController` ;
- `LaboOptionsTabController` ;
- `LPDCTabController` ;
- `TabSummaryController` ;
- `MachineSearchController`.

4.3 Modele

`src/model.py` contient deux classes principales :

- `Project` : etat metier courant, calculs, regroupements, exports ;
- `Model` : enveloppe `QObject` qui expose les signaux Qt et la serialisation.

Signaux principaux :

- `project_changed` : reconstruction des onglets apres changement structurant ;
- `data_updated` : simple rafraichissement des affichages et totaux.

4.4 Chargement des donnees

`src/utils/ApplicationData.py` :

- lit `config.yaml` ;

- charge tous les fichiers JSON de reference ;
- transforme ces donnees en objets Python utilisables par l'application ;
- charge egalement la feuille de style QSS.

4.5 Vue principale

`src/view.py` fournit `MainWindow`, une fenetre Qt qui contient simplement un `QTabWidget`.

4.6 Composants transverses

- `src/utils/BaseTaskTabController.py` : logique commune des onglets de taches ;
 - `src/utils/TabTasks.py` : tableau de taches, categories repliables, corrections ;
 - `src/utils/widgets.py` : widgets Qt personnalisés ;
 - `src/utils/exports.py` : exports Excel et export rapide ;
 - `src/utils/MachineDatabase.py` : chargement, recherche et ecriture dans la base REX.
-

5. Modele metier et regles de calcul

5.1 Contexte de calcul

Le calcul d'heures repose sur le contexte courant du projet :

- `product`
- `machine_type`
- `affaire`
- `secteur`
- coefficients calculs
- coefficients options
- coefficients LPDC
- coefficient labo par affaire

5.2 Types de taches

Toutes les taches heritent de `AbstractTask` dans `src/utils/Task.py`.

GeneralTask

Represente les taches generales d'ingenierie.

- base dependante du produit ;
- coefficient affaire ;
- coefficient secteur ;
- indicateur `multiplicative` pour les affaires multi-machines ;
- repartition ORTEMS par code job.

Calcul

Represente un calcul technique.

- heures par type de machine ;
- statut **mandatory** ou **optional** selon la machine ;
- coefficient par categorie dependant du type d'affaire.

Option

Represente une option technique selectionnable.

- active uniquement si cochee ;
- coefficient par categorie dependant du type d'affaire.

LPDCDocument

Represente un document contractuel ou technique.

- actif si la machine est applicable ;
- obligatoire pour certains secteurs ;
- optionnel sinon selon **option_possible** ;
- coefficient secteur x coefficient affaire.

Labo

Represente une tache de laboratoire.

- peut etre obligatoire selon le secteur ;
- sinon selectable ;
- coefficient secteur x coefficient affaire.

5.3 Totaux

Le cycle de calcul est le suivant :

1. somme des heures effectives de toutes les familles ;
2. application du pourcentage **Divers risques techniques** ;
3. ajout du supplement multi-machines sur les seules taches generales multiplicatives ;
4. application du REX par coefficient ou par valeur finale saisie.

Regle multi-machines :

- 1 machine : pas de supplement ;
- 2 machines : 100 % sur la machine supplementaire ;
- 3 a 5 machines : 75 % par machine supplementaire ;
- 6 a 25 machines : 35 % par machine supplementaire ;
- plus de 25 machines : 15 % par machine supplementaire.

5.4 Delai d'etude

Le delai d'etude est derive de la repartition ORTEMS, en particulier des heures affectees a **PROJ_MACHINE_DEF**.

Les parametres utilises sont stockes dans **data/base_data.json** :

- `n_projeteurs` par secteur ;
 - `taux_productivite` ;
 - `pct_conges` ;
 - `demarrage_mois`.
-

6. Configuration et donnees

6.1 Fichier de configuration

`config.yaml` contient :

- les chemins des donnees JSON ;
- le chemin de la base REX Excel ;
- les chemins des templates Excel ;
- les dossiers d'import, de sauvegarde et d'export rapide ;
- les parametres d'interface : theme, stylesheet, titre, taille de fenetre.

6.2 Fichiers de donnees

Le dossier `data/` contient les regles metier.

- `base_data.json` : personnes, types produit, produits, DAS, secteurs, jobs, parametres de delai ;
- `general_task_data_new.json` : taches generales hierarchiques ;
- `calculs.json` : categories, coefficients affaire, liste des calculs ;
- `options.json` : categories, coefficients affaire, liste des options ;
- `LPDC.json` : categories, coefficients LPDC, documents ;
- `labo.json` : categories et coefficients du laboratoire.

Dans la plupart des cas, une evolution metier se fait d'abord dans ces JSON, pas dans le code Python.

6.3 Assets et templates

- `assets/Affaires/` : projets sauvegardes ou imports de travail ;
 - `assets/Marine ref/` : references marine au format JSON ;
 - `template/ortems_template.xlsx` : template ORTEMS ;
 - `template/chiffrage_template.xlsx` : template du rapport Excel.
-

7. Format des projets sauvegardes

La sauvegarde JSON enregistre :

- les champs scalaires du projet ;
- uniquement les modifications par rapport aux donnees de reference ;
- les corrections de categorie.

Structure generale :


```
{
  "version": 1,
  "project": {
    "crm_number": "...",
    "client": "...",
    "affaire": "...",
    "das": "...",
    "secteur": "...",
    "machine_type": "...",
    "product": "..."
  },
  "modifications": {
    "lpdc_docs": [],
    "options": [],
    "calculs": [],
    "tasks": [],
    "labo": [],
    "category_corrections": {}
  }
}
```

Points a noter :

- les objets sont identifiés par leur **index** ;
- les selections et corrections manuelles sont restaurees au chargement ;
- la valeur finale **manual_rex_hours** n'est pas serialisee comme champ distinct : la sauvegarde conserve le coefficient REX equivalent.

8. Exports

8.1 Export JSON

Sauvegarde du projet courant pour reprise ulterieure dans l'application.

8.2 Export ORTEMS

Produit un fichier Excel a partir de **template/ortems_template.xlsx**.

Le calcul s'appuie sur la repartition ORTEMS generee par **Project.make_ortems_repartition()**.

8.3 Rapport Excel

Produit un rapport de chiffrage detaille a partir de **template/chiffrage_template.xlsx**.

Le rapport :

- injecte l'en-tete projet ;
- ecrit les heures par famille ;
- insere dynamiquement des lignes pour certaines sections ;
- applique le coefficient REX courant dans la colonne finale.

8.4 Export rapide

L'export rapide genere en une seule action :

- le JSON du projet ;
- le rapport Excel ;
- le fichier ORTEMS.

Les fichiers sont ecrits dans `quick-export-path` et `project-save-dir`.

9. Structure du depot

Structure logique principale :

```
HET_3/  
|-- main.py  
|-- config.yaml  
|-- build.bat  
|-- requirements.txt  
|-- data/  
|-- assets/  
|-- template/  
`-- src/  
    |-- controller.py  
    |-- model.py  
    |-- view.py  
    |-- styles.qss  
    |-- tabs/  
    `-- utils/
```

Fichiers Python les plus importants :

- `src/controller.py` : orchestration generale ;
 - `src/model.py` : etat projet, calculs, serialisation ;
 - `src/utils/ApplicationData.py` : chargement de la configuration et des JSON ;
 - `src/utils/Task.py` : hierarchie metier des taches ;
 - `src/utils/exports.py` : exports Excel ;
 - `src/utils/MachineDatabase.py` : moteur de recherche REX.
-

10. Guide de maintenance

Pour faire evoluer l'application proprement :

Ajouter ou modifier une regle metier

Commencer par verifier si la modification releve d'un fichier JSON dans `data/`. C'est le cas de la plupart des taches, coefficients, categories et repartitions ORTEMS.

Modifier le comportement d'un onglet

Regarder d'abord dans `src/tabs/`, puis dans `src/utils/BaseTaskTabController.py` et `src/utils/TabTasks.py` pour la logique commune.

Modifier un calcul global

Les points d'entree sont principalement dans `Project` :

- `apply_defaults()` ;
- `compute_first_machine_subtotal()` ;
- `compute_first_machine_total()` ;
- `compute_n_machines_total()` ;
- `calculate_total_with_rex()` ;
- `make_ortems_repartition()` ;
- `compute_delai_etude()`.

Modifier le look and feel

Le style applicatif est charge depuis `src/styles.qss` via `config.yaml`.

11. Resume

Chiffrage HET est une application de chiffrage pilotée par des données JSON, organisée autour d'un modèle `Project`, de tableaux de tâches génériques et d'un onglet de synthèse qui centralise calcul, délai et exports.

La clé pour maintenir le projet sans casser le comportement est de distinguer clairement :

- ce qui relève des données métier dans `data/` ;
- ce qui relève du calcul dans `src/model.py` et `src/utils/Task.py` ;
- ce qui relève de l'interface dans `src/tabs/` et `src/utils/TabTasks.py`.